

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Nichenametla Venkata Mani
Chandan
192525392RC

```
d -> 1
e -> 1
h -> 1
l -> 3
o -> 2
r -> 1
w -> 1
```

Your Code

```
1 s = input().strip()
2
3 s = s.replace(" ", "").lower()
4
5 freq = {}
6 for ch in s:
7     freq[ch] = freq.get(ch, 0) + 1
8
9 for ch in sorted(freq.keys()):
10    print(f"{ch} -> {freq[ch]}")
```

Input

Hello World

Output

```
d -> 1
e -> 1
h -> 1
l -> 3
o -> 2
r -> 1
w -> 1
```

✓ Submitted

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Nichenametla Venkata Mani
Chandan
192525392RC

Your Code

```
1 def analyze_sentence(sentence: str):
2     vowels_set = set("aeiou")
3     sentence = sentence.lower()
4
5     vowels_count = 0
6     consonants_count = 0
7
8     vowels_freq = {} # vowel -> count
9     consonants_freq = {} # consonant -> count
10
11     for ch in sentence:
12         if ch.isdigit() or ch == ' ':
13             continue
14
15         if ch.isalpha():
16             if ch in vowels_set:
17                 vowels_count += 1
18                 vowels_freq[ch] = vowels_freq.get(ch, 0) + 1
19             else:
20                 consonants_count += 1
21                 consonants_freq[ch] = consonants_freq.get(ch, 0) + 1
22
23     # Most frequent vowel
24     if vowels_freq:
25         most_freq_vowel = max(vowels_freq, key=vowels_freq.get)
26         most_vowel_count = vowels_freq[most_freq_vowel]
27     else:
```

Input

Python Programming

Output

Vowels: 4 | Consonants: 13 Most
frequent vowel: o (2) Least
frequent consonant: y (1) Vowel-to-
Consonant Ratio: 0.31

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Nichenametla Venkata Mani
Chandan
192525392RC



```
28     most_freq_vowel = 'N/A'
29     most_vowel_count = 0
30
31     # Least frequent consonant
32     if consonants_freq:
33         least_freq_consonant = min(consonants_freq, key=consonants_freq.get)
34         least_consonant_count = consonants_freq[least_freq_consonant]
35     else:
36         least_freq_consonant = 'N/A'
37         least_consonant_count = 0
38
39     # Ratio vowels to consonants rounded to 2 places
40     if consonants_count == 0:
41         ratio = 0.00
42     else:
43         ratio = round(vowels_count / consonants_count, 2)
44
45     return (vowels_count, consonants_count,
46           most_freq_vowel, most_vowel_count,
47           least_freq_consonant, least_consonant_count,
48           ratio)
49
50
51 def main():
52     sentence = input().rstrip("\n")
53     (vowels_count, consonants_count,
54     most_vowel, most_vowel_count,
55     least_consonant, least_consonant_count,
56     ratio) = analyze_sentence(sentence)
```

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Nichenametla Venkata Mani
Chandan
192525392RC

```
45     return (vowels_count, consonants_count,  
46             most_freq_vowel, most_vowel_count,  
47             least_freq_consonant, least_consonant_count,  
48             ratio)  
49  
50  
51 def main():  
52     sentence = input().rstrip("\n")  
53     (vowels_count, consonants_count,  
54     most_vowel, most_vowel_count,  
55     least_consonant, least_consonant_count,  
56     ratio) = analyze_sentence(sentence)  
57  
58     print(f"Vowels: {vowels_count} | Consonants: {consonants_count} "  
59           f"Most frequent vowel: {most_vowel} ({most_vowel_count}) "  
60           f"Least frequent consonant: {least_consonant} ({least_consonant_count}) "  
61           f"Vowel-to-Consonant Ratio: {ratio:.2f}")  
62  
63  
64 if __name__ == "__main__":  
65     main()
```

✓ Submitted

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Nichenametla Venkata Mani
Chandan
192525392RC



Your Code

```
1 def compress_rle(s: str) -> str:
2     if not s:
3         return ""
4
5     n = len(s)
6     i = 0
7     encoded_parts = []
8
9     while i < n:
10        ch = s[i]
11        j = i
12        while j < n and s[j] == ch:
13            j += 1
14        count = j - i
15
16        if count == 1:
17            encoded_parts.append(ch)
18        else:
19            encoded_parts.append(f"{ch}{count}")
20
21        i = j
22
23    compressed = "".join(encoded_parts)
24    # If compressed is not shorter, return original string
25    return compressed if len(compressed) < len(s) else s
26
```

Input

aabbccddddee

Output

Compressed: a2b3c2d4e2
Decompressed: aabbccddddee

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Nichenametla Venkata Mani
Chandan
192525392RC



```
27
28 def decompress_rle(encoded: str) -> str:
29     if not encoded:
30         return ""
31
32     i = 0
33     n = len(encoded)
34     result = []
35
36     while i < n:
37         ch = encoded[i]
38         i += 1
39
40         # collect digits after the character (if any)
41         if i < n and encoded[i].isdigit():
42             num_start = i
43             while i < n and encoded[i].isdigit():
44                 i += 1
45             count = int(encoded[num_start:i])
46             result.append(ch * count)
47         else:
48             # single character without count
49             result.append(ch)
50
51     return "".join(result)
52
53
54 def rle_encode_decode(s: str):
55     compressed = compress_rle(s)
```

```

46         result.append(ch * count)
47     else:
48         # single character without count
49         result.append(ch)
50
51     return "".join(result)
52
53
54 def rle_encode_decode(s: str):
55     compressed = compress_rle(s)
56     decompressed = decompress_rle(compressed)
57     return compressed, decompressed
58
59
60 # ----- Input/Output handling for online judge -----
61 if __name__ == "__main__":
62     s = input().rstrip("\n")
63     compressed, decompressed = rle_encode_decode(s)
64
65     print("Compressed:", compressed)
66     print("Decompressed:", decompressed)

```

✔ Submitted

SIMATS Online Programming Lab

PYTHON PROGRAMMING



Nichenametla Venkata Mani
Chandan
192525392RC



Your Code

```
1 import re
2 from collections import Counter
3
4 STOP_WORDS = {"the", "is", "a", "an", "in", "of", "and", "to", "for"}
5
6 def is_sentence_end(ch: str) -> bool:
7     return ch in ".?! "
8
9 def tokenize_words(text: str):
10     # Words with optional attached punctuation (we strip punctuation later)
11     # Keep only alphabetic characters as the word core
12     raw_tokens = re.findall(r"[A-Za-z']+", text)
13     return raw_tokens
14
15 def split_paragraphs(text: str):
16     # Paragraphs separated by one or more blank lines
17     parts = re.split(r'\n\s*\n+', text.strip())
18     # If user wrote no blank lines, treat whole text as one paragraph
19     return [p.strip() for p in parts if p.strip()]
20
21 def count_sentences(text: str):
22     # Sentence ends with . ! ?
23     # Handle cases with multiple end marks by counting each end char
24     count = 0
25     for ch in text:
26         if is_sentence_end(ch):
```

Input

Python is a powerful programming
language

Output

```
Enter paragraph of text: Words: 6
| Sentences: 0 | Paragraphs: 1
Avg Word Length: 6.00 | Avg
Sentence Length: 0.00
Top 5 Words: python(1),
powerful(1), programming(1)
```


SIMATS Online Programming Lab

PYTHON PROGRAMMING



Nichenametla Venkata Mani
Chandan
192525392RC

```
26     if is_sentence_end(ch):
27         count += 1
28     return count
29
30 def normalize_word(word: str) -> str:
31     return word.lower()
32
33 def count_vowel_groups(word: str):
34     # Approximate syllables as number of vowel groups
35     # vowel groups: consecutive vowels count as one group
36     vowels = set("aeiouAEIOU")
37     w = word.lower()
38     groups = 0
39     in_vowel = False
40     for ch in w:
41         if ch in vowels:
42             if not in_vowel:
43                 groups += 1
44                 in_vowel = True
45         else:
46             in_vowel = False
47     return max(1, groups) # at least 1 syllable per word
48
49 def flesch_reading_ease(total_words, total_sentences, total_syllables)
50     # Flesch Reading Ease formula given:
51     # 206.835 - 1.015*(words/sentences) - 84.6*(syllables/words)
52     # Avoid division by zero
53     if total_words == 0:
54         return 0.0
```

```
55     if total_sentences == 0:
56         total_sentences = 1
57     return 206.835 - 1.015 * (total_words / total_sentences) - 84.6 *
58
59 def advanced_text_stats(paragraph_text: str):
60     text = paragraph_text.strip()
61
62     paragraphs = split_paragraphs(text)
63     paragraph_count = len(paragraphs)
64
65     sentence_count = count_sentences(text)
66
67     words_list = tokenize_words(text)
68     total_words = len(words_list)
69
70     # Average word length
71     if total_words > 0:
72         avg_word_length = sum(len(w) for w in words_list) / total_words
73     else:
74         avg_word_length = 0.0
75
76     # Average sentence length
77     if sentence_count > 0:
78         avg_sentence_length = total_words / sentence_count
79     else:
80         avg_sentence_length = 0.0
81
82     # Top 5 most frequent words excluding stop words
83     normalized = [normalize_word(w) for w in words_list]
```

```
85     freq = Counter(filtered)
86     top5 = freq.most_common(5)
87
88     # Syllable counting using vowel groups per word
89     total_syllables = sum(count_vowel_groups(w) for w in words_list)
90
91     # Flesch Reading Ease
92     score = flesch_reading_ease(total_words, sentence_count, total_syl
93
94     # Format output
95     top5_str = ", ".join([f"{w}({c})" for w, c in top5]) if top5 else
96
97     return {
98         "words": total_words,
99         "sentences": sentence_count,
100        "paragraphs": paragraph_count,
101        "avg_word_length": avg_word_length,
102        "avg_sentence_length": avg_sentence_length,
103        "top5": top5_str,
104        "flesch": round(score, 1) # rounding as typical
105    }
106
107 def main():
108     # Accept a paragraph of text
109     text = input("Enter paragraph of text: ")
110
111     result = advanced_text_stats(text)
112
113     print(f"Words: {result['words']} | Sentences: {result['sentences']")
```

```

96
97     return {
98         "words": total_words,
99         "sentences": sentence_count,
100        "paragraphs": paragraph_count,
101        "avg_word_length": avg_word_length,
102        "avg_sentence_length": avg_sentence_length,
103        "top5": top5_str,
104        "flesch": round(score, 1) # rounding as typical
105    }
106
107 def main():
108     # Accept a paragraph of text
109     text = input("Enter paragraph of text: ")
110
111     result = advanced_text_stats(text)
112
113     print(f"Words: {result['words']} | Sentences: {result['sentences']}")
114     print(f"Avg Word Length: {result['avg_word_length']:.2f} | Avg Sen")
115     print(f"Top 5 Words: {result['top5']}")
116     print(f"Flesch Reading Ease: {result['flesch']}")
117
118 if __name__ == "__main__":
119     main()

```

✓ Submitted